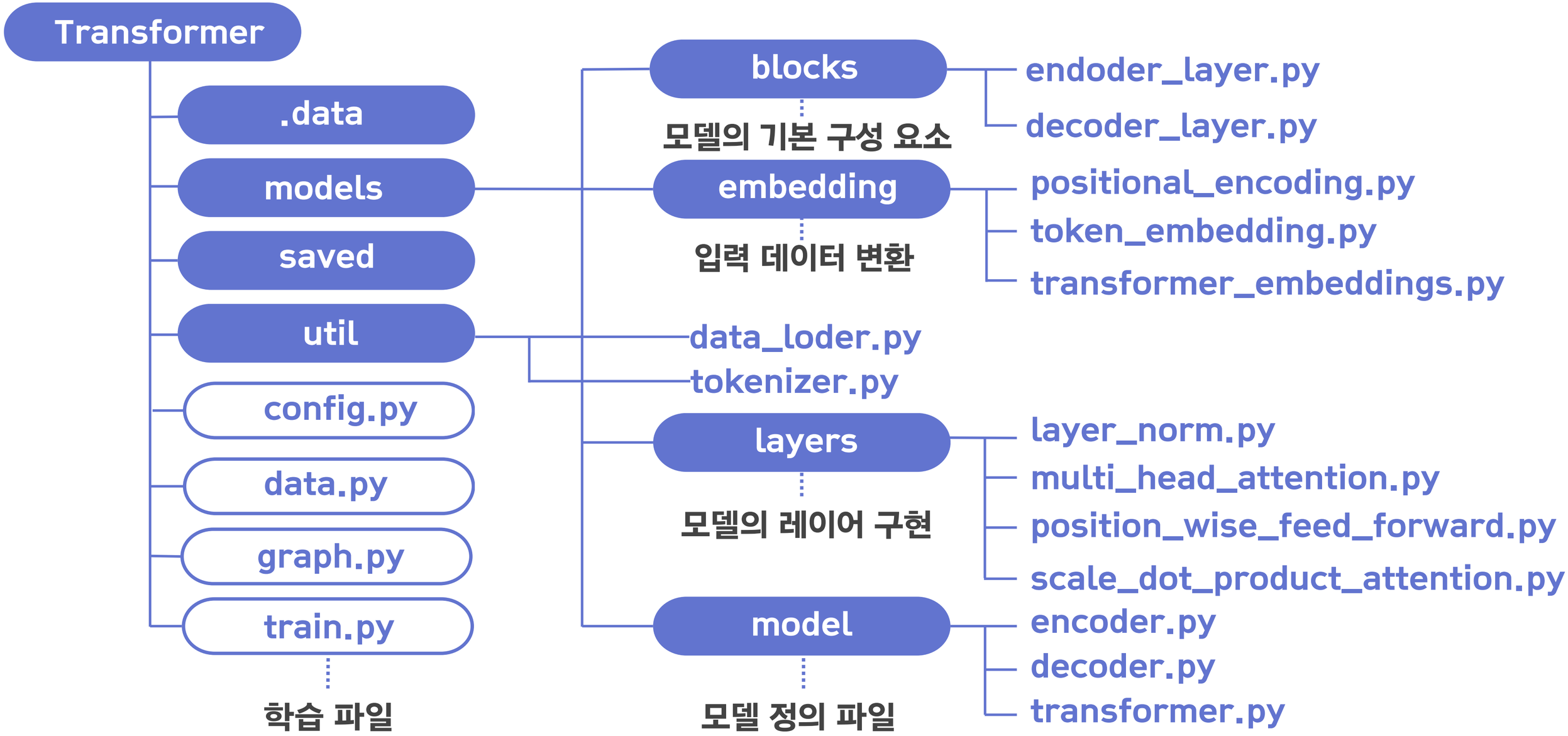
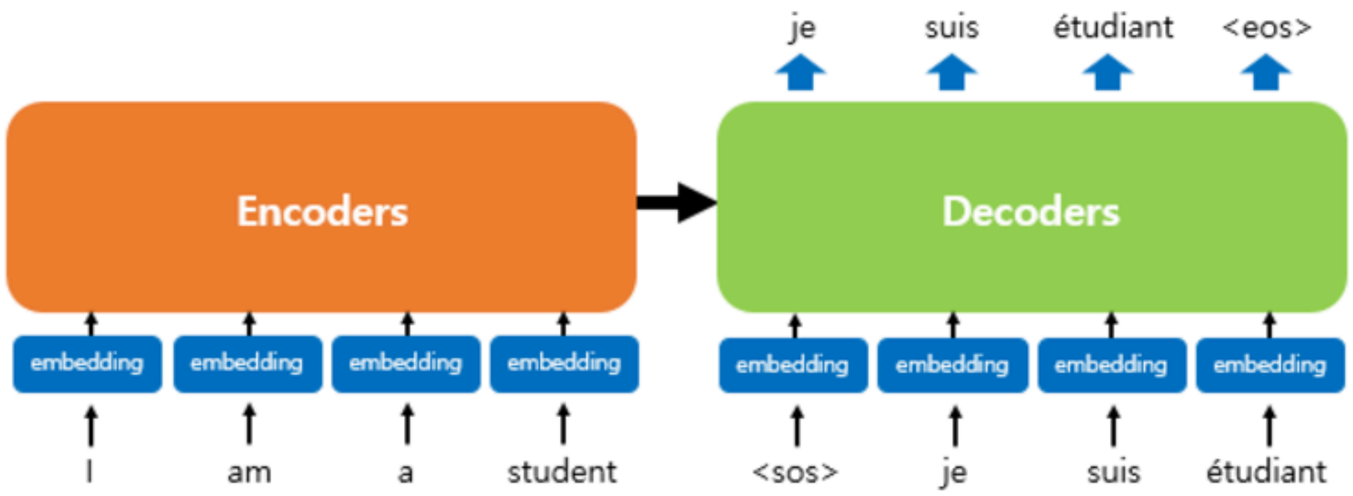
Transformer 코드 분석

2022111723 데이터사이언스학과 김경은

01 Transformer 소스 코드 구조



02 Transformer 모델



Transformer 주요 하이퍼파라미터

batch_size = 128 : 1번의 학습에 사용되는 문장 수

max_len = 256 : 한 문장내 단어 최대 갯수

d_model = 512 : 한 단어의 벡터 크기

n_layers = 6 : 레이어 갯수

n_heads = 8 : Multi Head Attention내 Scaled Dot Attention 갯수

ffn_hiddn = 2048 : FF 망 크기

drop_prob = 0.1

Transformer 모델 정의

Transformer 모델은 자연어 처리 분야에서 기계 번역, 텍스트 생성, 질문 응답 등의 작업에 높은 성능을 보인다.

Transformer 모델 구성 요소

Transformer 모델은 크게 입력 시퀀스를 처리하는 **Encoder**, 출력 시퀀스를 생성하는 **Decoder**로 이루어져 있다. 인코더는 소스 시퀀스의 정보를 압축해 디코더로 보내주며 이 과정을 **인코딩**이라고 한다. 디코더는 인코더가 보내준 소스 시퀀스 정보를 받아서 타깃 시퀀스를 생성하며 이 과정을 **디코딩**이라고 한다.

인코더가 소스 시퀀스를 압축해 디코더로 보내고 디코더는 인코더에서 보내온 정보와 현재 디코더 입력을 모두 감안해 다음 토큰을 맞힌다.트랜스포머의 학습은 인코더와 디코더 입력이 주어졌을 때 **정답에 해당하는 단어의 확률 값을 높이는 방식**으로 수행된다.

Encoder

Encoder는 입력 시퀀스를 처리하는 부분으로, 여러 개의 Encoder Layer로 구성된다.

Decoder

Decoder는 출력 시퀀스를 생성하는 부분으로, 여러 개의 Decoder Layer로 구성된다.

03 Transformer 구조

인코더에서 다음과 같은 요소를 반복하여 블록을 수십 개 쌓아서 구성한다.

인코더 블록의 구성 요소

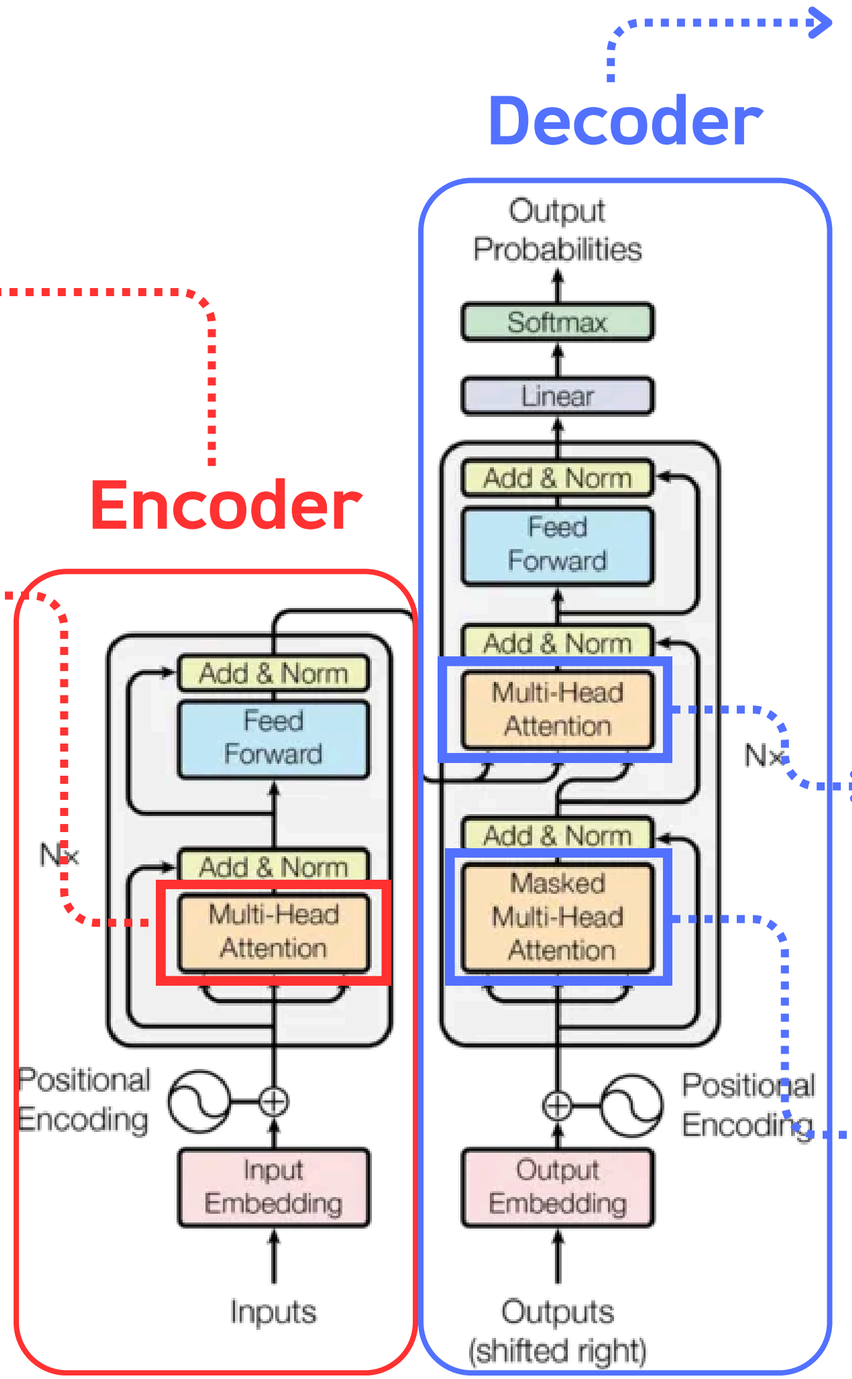
- 멀티 헤드 어텐션
- 피드 포워드 뉴럴네트워크
- 잔차 연결 및 레이어 정규화

Encoder Self-Attention

트랜스포머의 인코더에는 n_layers만큼의 인코더 층이 있는데, 하나의 층은 크게 셀프 어텐션과 피드 포워드 신경망으로 나뉘어진다.

- 셀프 어텐션을 병렬적으로 사용한게 멀티 헤드 셀프 어텐션

셀프 어텐션은 Key, Value, Query가 모두 소스 시퀀스와 관련된 정보로, 각 입력 단어마다 q, k, v 벡터를 얻는다. 유사도 점수 계산, 가중치 정규화, 가중치 적용을 통해 입력 시퀀스의 모든 토큰 간의 관계를 계산해 각 단어의 표현을 업데이트 한다.



디코더 또한 인코더와 유사하지만 **마스크를 적용한 멀티 헤드 어텐션**이 차이점이 있다. 또한 인코더가 보내온 정보와 디코더 입력을 함께 이용해 멀티 헤드 어텐션을 수행하는 모듈이 추가되었다.

디코더 블록의 구성 요소

- 마스크를 적용한 멀티 헤드 어텐션
- 멀티 헤드 어텐션
- 피드포워드 뉴럴 네트워크
- 잔차 연결 및 레이어 정규화

Encoder - Decoder Attention

Decoder가 Encoder에서 처리된 입력 시퀀스의 정보를 기반으로 Decoder의 예측 능력을 강화한다.

Masked Decoder Self-Attention

Decoder가 현재 예측할 단어를 기준으로 이전 위치의 정보만을 참고하도록 하여 생성 과정에서 순차성을 보장한다.

04 Transformer 입력

```python
class TokenEmbedding(nn.Embedding):
    def __init__(self, vocab_size, d_model):
        super(TokenEmbedding, self).__init__(vocab_size, d_model, padding_idx=1)
```

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, device):
        super(PositionalEncoding, self).__init__()

        # same size with input matrix (for adding with input matrix)
        self.encoding = torch.zeros(max_len, d_model, device=device)
        self.encoding.requires_grad = False  # we don't need to compute gradient

        pos = torch.arange(0, max_len, device=device)
        pos = pos.float().unsqueeze(dim=1)
        # 1D => 2D unsqueeze to represent word's position

        _2i = torch.arange(0, d_model, step=2, device=device).float()
        # 'i' means index of d_model (e.g. embedding size = 50, 'i' = [0,50])
        # "step=2" means 'i' multiplied with two (same with 2 * i)

        self.encoding[:, 0::2] = torch.sin(pos / (10000 ** (_2i / d_model)))
        self.encoding[:, 1::2] = torch.cos(pos / (10000 ** (_2i / d_model)))
        # compute positional encoding to consider positional information of words

    def forward(self, x):
        # self.encoding
        # [max_len = 512, d_model = 512]
        batch_size, seq_len = x.size()
        # [batch_size = 128, seq_len = 30]
        return self.encoding[:seq_len, :]
        # [seq_len = 30, d_model = 512]
        # it will add with tok_emb : [128, 30, 512]
```

Token Embedding

토큰 임베딩은 **텍스트 데이터의 각 토큰을 고차원 벡터로 변환**하는 과정이다. 초기화 메서드를 통해 임베딩 매트릭스를 생성하고, 패딩 토큰을 설정하여 **유사한 의미를 가진 토큰들을 유사한 벡터로 표현**한다.

Positional Encoding

Transformer는 단어의 위치 정보를 얻기 위해서 **각 단어의 임베딩 벡터에 위치 정보들을 더하여** 모델의 입력으로 사용하는데, 이를 포지셔널 인코딩이라고 한다.

Transformer는 사인 함수와 코사인 함수의 값을 임베딩 벡터에 더해주므로서 단어의 순서 정보를 더해준다.

- self.encoding[:, 0::2]는 짝수 인덱스에 대해 사인 함수를 적용
- self.encoding[:, 1::2]는 홀수 인덱스에 대해 코사인 함수를 적용

forward 메서드는 입력 x를 받아, 포지셔널 인코딩을 반환한다.

- self.encoding[:seq_len, :]는 입력 시퀀스 길이(seq_len)에 맞게 포지셔널 인코딩을 잘라냄

이를 통해 각 임베딩 벡터에 포지셔널 인코딩의 값을 더하면 **같은 단어라고 하더라도 문장 내의 위치에 따라서 트랜스포머에 들어가는 임베딩 벡터의 값이 달라진다.**

05 Multi-Head Attention

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_head):
        super(MultiHeadAttention, self).__init__()
        self.n_head = n_head
        self.attention = ScaleDotProductAttention()
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_concat = nn.Linear(d_model, d_model)

    def forward(self, q, k, v, mask=None):
        q, k, v = self.w_q(q), self.w_k(k), self.w_v(v)
        q, k, v = self.split(q), self.split(k), self.split(v)
        out, attention = self.attention(q, k, v, mask=mask)
        out = self.concat(out)
        out = self.w_concat(out)
        return out

    def split(self, tensor):
        batch_size, length, d_model = tensor.size()
        d_tensor = d_model // self.n_head
        tensor = tensor.view(batch_size, length, self.n_head, d_tensor).transpose(1, 2)
        # it is similar with group convolution (split by number of heads)
        return tensor

    def concat(self, tensor):
        batch_size, head, length, d_tensor = tensor.size()
        d_model = head * d_tensor
        tensor = tensor.transpose(1, 2).contiguous().view(batch_size, length, d_model)
        return tensor
```

q, k, v는 각각 선형 변환된다.

d_model의 차원을 n_heads개로 나누어 512/8의 차원을 가지는 q, k, v에 대해 n_heads개의 병렬 어텐션을 수행한다.

-> 8개의 Scaleddotproduct attention 을 동시에 연산

어텐션 출력을 다시 결합하고 최종 선형 변환을 거친다.

n_heads 개수 만큼 텐서를 split하는 함수

분할된 텐서를 다시 결합한다.

- 인코더 레이어가 다양한 관점에서 학습한 정보를 통합하여 다음 인코더 레이어로 전달
  -> 인코더에서의 입력 크기가 출력에서도 동일 크기로 유지되어야 다음 인코더에서도 다시 입력이 가능

06 Scale Dot Product Attention

```python
class ScaleDotProductAttention(nn.Module):
    def __init__(self):
        super(ScaleDotProductAttention, self).__init__()
        self.softmax = nn.Softmax(dim=-1)

    def forward(self, q, k, v, mask=None, e=1e-12):
        batch_size, head, length, d_tensor = k.size()
        k_t = k.transpose(2, 3)  # transpose
        score = (q @ k_t) / math.sqrt(d_tensor)
        if mask is not None:
            score = score.masked_fill(mask == 0, -10000)
        score = self.softmax(score)
        v = score @ v
        return v, score
```

$$softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention은 입력 데이터를 n_head로 분할하고 독립적으로 Self-Attention을 수행한다. 이를 통해 각 헤드가 서로 다른 관계와 패턴을 학습할 수 있어 모델의 표현력을 향상시킬 수 있다.
트랜스포머는 어텐션을 내적만을 사용하는 어텐션 함수에서 특정값으로 나눠준 어텐션함수를 사용한다. 여기서 값을 스케일링하는 것을 추가하여 스케일닷 프로덕트 어텐션을 사용한다.
이 과정을 행렬 연산을 사용해 일괄 처리를 할 수 있다. **즉, 어텐션 스코어 행렬에 소프트맥스 함수를 사용하고 V행렬을 곱하면 각 단어의 어텐션 값을 모두 가지는 어텐션 값 행렬이 결과로 도출된다.**
즉, Multi-Head Attention은 여러 Scale Dot-Product Attention을 병렬로 실행하고 이를 결합해 더 복잡한 관계를 학습한다.

```python
class PositionwiseFeedForward(nn.Module):

    def __init__(self, d_model, hidden, drop_prob=0.1):
        super(PositionwiseFeedForward, self).__init__()
        self.linear1 = nn.Linear(d_model, hidden)
        self.linear2 = nn.Linear(hidden, d_model)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(p=drop_prob)

    def forward(self, x):
        x = self.linear1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.linear2(x)
        return x
```

07 Position Wise Feed Forward

PositionWiseFeedForward는 인코더와 디코더에서 공통적으로 가지고 있는 서브층으로 각 위치의 벡터를 독립적으로 변환해 더 복잡한 표현을 학습한다.

인코더의 입력을 벡터 단위로 봤을 때, 각 벡터들이 멀티 헤드 어텐션 층을 지나 PositionWiseFeedForward를 통과한다. 최종 출력은 인코더의 입력의 크기가 보존되어 다음 인코더 층으로 전달되고 다음 층에서도 동일한 인코더 연산이 반복된다.

x : 멀티 헤드 어텐션 출력 결과로 나온 행렬

1. 선형 변환 : 입력 벡터를 은닉층의 차원으로 변환
2. 활성화 함수 : ReLU 활성화 함수를 적용하여 비선형성을 도입
3. Dropout : 과적합을 방지
4. 선형 변환 : 은닉층의 차원을 다시 입력 벡터의 차원으로 변환한다.

```python
class LayerNorm(nn.Module):
    def __init__(self, d_model, eps=1e-12):
        super(LayerNorm, self).__init__()
        self.gamma = nn.Parameter(torch.ones(d_model))
        self.beta = nn.Parameter(torch.zeros(d_model))
        self.eps = eps

    def forward(self, x):
        mean = x.mean(-1, keepdim=True)
        var = x.var(-1, unbiased=False, keepdim=True)
        # '-1' means last dimension.

        out = (x - mean) / torch.sqrt(var + self.eps)
        out = self.gamma * out + self.beta
        return out
```

08 Layer Norm

트랜스포머에서는 두 개의 서브층을 가진 인코더에 추가적으로 사용하는 기법으로 Layer Normalization을 사용한다.

텐서의 마지막 차원에 대해 평균과 분산을 구하고 정규화하여 학습을 돕는다.

입력 데이터 x에서 평균을 뺀 후 분산의 제곱근으로 나누어 **정규화** 한다.

- self.eps를 통해 분모가 0이 되는 것을 방지

- Gamma : **Rescalinig**, 값의 크기를 1에 가깝게 조정
- Beta : **Shift**, 중심축을 0에 가깝게 조정

09 Encoder Layer

```python
class EncoderLayer(nn.Module):

    def __init__(self, d_model, ffn_hidden, n_head, drop_prob):
        super(EncoderLayer, self).__init__()
        self.attention = MultiHeadAttention(d_model=d_model, n_head=n_head)
        self.norm1 = LayerNorm(d_model=d_model)
        self.dropout1 = nn.Dropout(p=drop_prob)

        self.ffn = PositionwiseFeedForward(d_model=d_model, hidden=ffn_hidden, drop_prob=drop_prob)
        self.norm2 = LayerNorm(d_model=d_model)
        self.dropout2 = nn.Dropout(p=drop_prob)

    def forward(self, x, src_mask):
        # 1. compute self attention
        _x = x
        x = self.attention(q=x, k=x, v=x, mask=src_mask)
```

→ 입력 데이터 x에 대해 멀티 헤드 어텐션 계산

```python
        # 2. add and norm
        x = self.dropout1(x)
        x = self.norm1(x + _x)
```

→ 첫 번째 레이어 정규화 적용

```python
        # 3. positionwise feed forward network
        _x = x
        x = self.ffn(x)
```

→ 포지션 와이즈 피드포워드 네트워크 적용

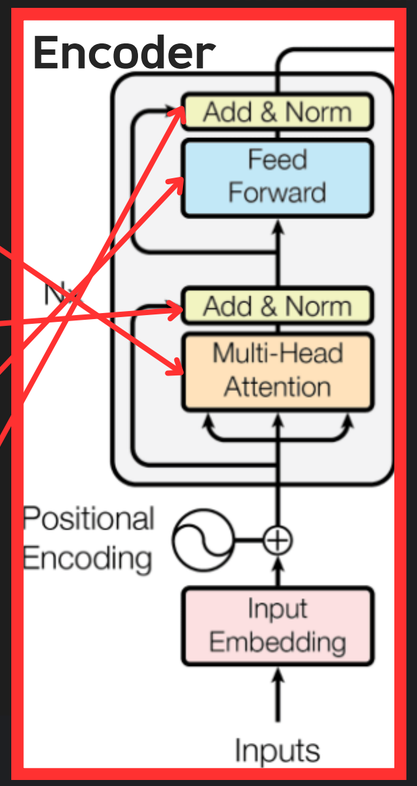
```python
        # 4. add and norm
        x = self.dropout2(x)
        x = self.norm2(x + _x)
        return x
```

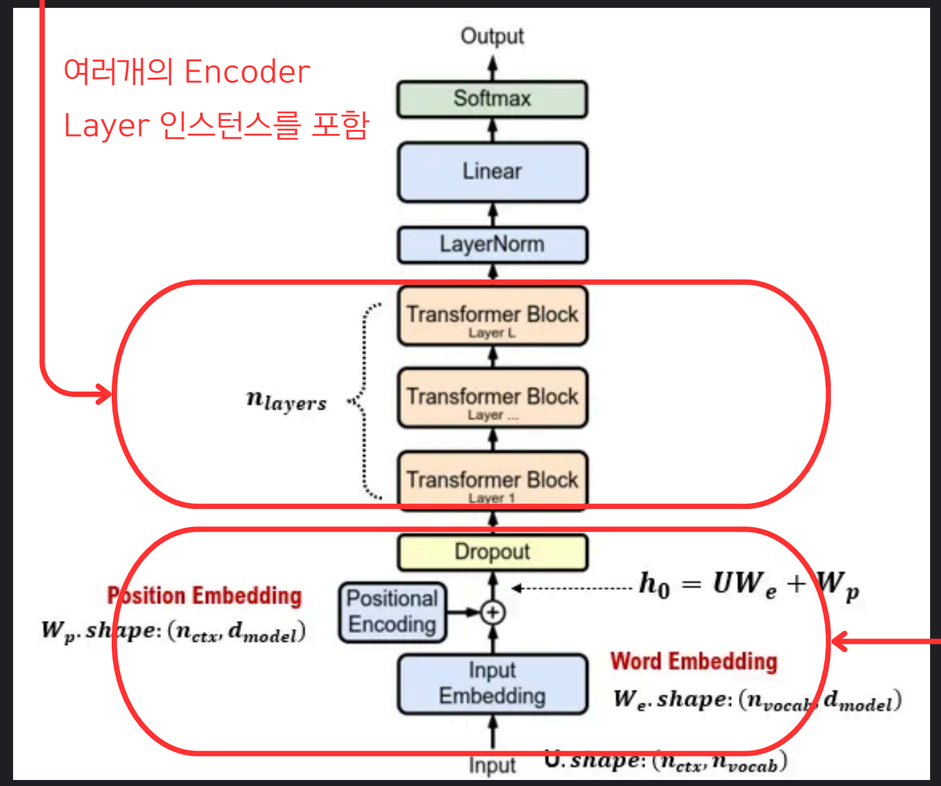
→ 두번 째 레이어 정규화 적용

```python
class Encoder(nn.Module):

    def __init__(self, enc_voc_size, max_len, d_model, ffn_hidden, n_head, n_layers, drop_prob, device):
        super().__init__()
        self.emb = TransformerEmbedding(d_model=d_model,
                                        max_len=max_len,
                                        vocab_size=enc_voc_size,
                                        drop_prob=drop_prob,
                                        device=device,
                                        )

        self.layers = nn.ModuleList([EncoderLayer(d_model=d_model,
                                                  ffn_hidden=ffn_hidden,
                                                  n_head=n_head,
                                                  drop_prob=drop_prob)
                                     for _ in range(n_layers)])
```

트랜스포머 임베딩 레이어

- 입력 시퀀스를 임베딩 벡터로 변환





10 Decoder Layer

```python
class DecoderLayer(nn.Module):

    def __init__(self, d_model, ffn_hidden, n_head, drop_prob):
        super(DecoderLayer, self).__init__()
        self.self_attention = MultiHeadAttention(d_model=d_model, n_head=n_head)
        self.norm1 = LayerNorm(d_model=d_model)
        self.dropout1 = nn.Dropout(p=drop_prob)

        self.enc_dec_attention = MultiHeadAttention(d_model=d_model, n_head=n_head)
        self.norm2 = LayerNorm(d_model=d_model)
        self.dropout2 = nn.Dropout(p=drop_prob)

        self.ffn = PositionwiseFeedForward(d_model=d_model, hidden=ffn_hidden, drop_prob=drop_prob)
        self.norm3 = LayerNorm(d_model=d_model)
        self.dropout3 = nn.Dropout(p=drop_prob)

    def forward(self, dec, enc, trg_mask, src_mask):
        # 1. compute self attention
        # 2. add and norm
        if enc is not None:
            _x = x
            x = self.enc_dec_attention(q=x, k=enc, v=enc, mask=src_mask)
            x = self.dropout2(x)
            x = self.norm2(x + _x)

        # 5. positionwise feed forward network
        # 6. add and norm
        return x
```

인코더에서 마지막 층의 인코더의 출력을 디코더에게 전달한다. 디코더 또한 n_layers만큼의 연산을 하는데, 이때마다 인코더가 보낸 출력을 각 디코더 층 연산에 사용한다.

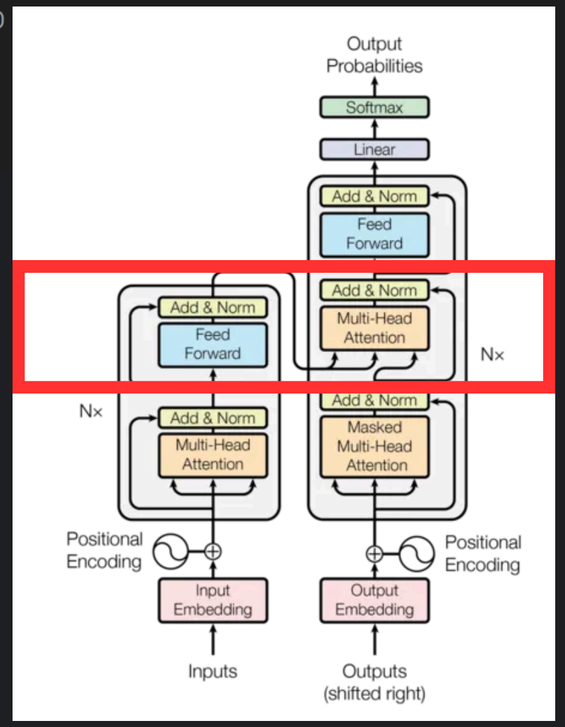
디코더는 총 세 개의 서브층으로 구성되어 첫 번째 서브층은 마스크드 멀티 헤드 어텐션이 수행된다.

- 세 개의 서브층 모두 서브층 연산 후에 인코더의 출력이 존재하는 경우에만 드롭아웃, 정규화가 수행된다.

```python
class Decoder(nn.Module):
    def __init__(self, dec_voc_size, max_len, d_model, ffn_hidden, n_head, n_layers, drop_prob, device):
        super().__init__()
        self.emb = TransformerEmbedding(d_model=d_model,
                                        drop_prob=drop_prob,
                                        max_len=max_len,
                                        vocab_size=dec_voc_size,
                                        device=device,
                                        )

        self.layers = nn.ModuleList([DecoderLayer(d_model=d_model,
                                                  ffn_hidden=ffn_hidden,
                                                  n_head=n_head,
                                                  drop_prob=drop_prob)
                                     for _ in range(n_layers)])

        self.linear = nn.Linear(d_model, dec_voc_size)

    def forward(self, trg, enc_src, trg_mask, src_mask):
        trg = self.emb(trg)

        for layer in self.layers:
            trg = layer(trg, enc_src, trg_mask, src_mask)

        # pass to LM head
        output = self.linear(trg)
        return output
```

최종 출력을 어휘 크기로 변환하는 선형 레이어

타겟 시퀀스 trg를 임베딩 벡터 변환, Decoder Layer를 거친 후 최종 출력을 어휘 크기로 변환



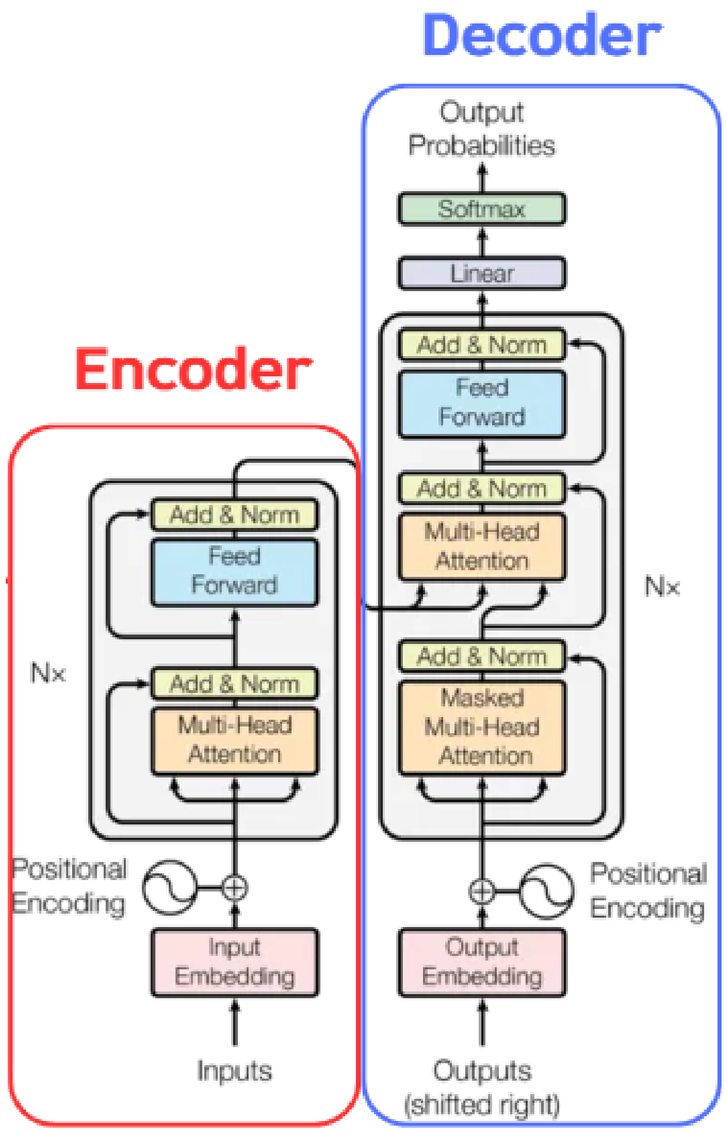
11 Transformer 작동 흐름 앞에서 구현한 인코더와 디코더 함수를 조합하여 트랜스포머 모델 구축

1. 입력 데이터

1. **입력 시퀀스 임베딩** : 입력 텍스트는 고유한 숫자로 변환된 후, 고차원 임베딩 벡터로 매핑되는데 벡터는 각 단어의 의미를 나타낸다.
2. **Positional Encoding 추가** : 트랜스포머 모델은 순서를 고려하지 않는 구조이기 때문에 Positional Encoding을 임베딩에 더해 시퀀스의 순서 정보를 포함한다. 이를 통해 입력 데이터가 순서를 고려한 상태로 Encodeer에 전달된다.

2. Encoder의 작동 흐름

1. **Multi-Head Attention** : 입력 데이터의 각 단어 간 관계를 계산하기 위해 Self-Attetion을 사용한다. 이를 통해 Query, Key, Value 벡터를 생성하고 단어 간 연관성을 계산한다. 병렬로 계산하여 다양한 관계를 포착한다.
2. **Add & Norm** : Self-Attetion의 출력에 입력을 더하고, Layer Normalization을 적용해 안정성을 높인다.
3. **Feed Forward Network** : 각 단어를 독립적으로 처리하는 두 개의 완전연결층을 통해 비선형 변환을 수행한다.
4. 위 단계를 여러 **Encoder Layer**에서 반복하며, 입력의 특징을 추출한다.



3. Decoder의 작동 흐름

1. **Multi-Head Attention (Self-Attention)** : 마스킹(현재 단어 이후의 위치는 가려 계산에 영향을 주지 않도록 함)을 사용해 아직 생성되지 않은 단어를 고려하지 않도록 한다.
2. **Multi-Head Attention (Encoder-Decoder Attention)** : Encoder의 출력을 Query로 사용해 입력과 출력 간 관계를 학습한다. 이를 통해 입력 데이터와 관련된 정보를 얻는다.
3. **Add & Norm, Feed Forward** : 인코더에서와 동일하게 적용
4. Decoder의 최종 출력은 **Softmax 함수**를 통해 확률 분포로 변환된다.

4. 반복적인 출력 생성

- 트랜스포머는 이전 단계에서 생성된 단어를 사용해 다음 단어를 반복적으로 예측한다.
- 이러한 과정은 원하는 길이의 출력 시퀀스가 생성될 때까지 반복된다.

12 Train

옵티마이저 파라미터 설정

init_lr = 1e-5 : Learning rate

factor = 0.9 : Epoc 당 Learning rate 감소 비율

adam_eps = 5e-9 : 분모가 0이 되지 않도록 제공되는 값

patience = 10 : Loss값 감소가 지정된 값 횟수동안 없으면 LR 감소

warmup = 100 : step이 warmup 전일때는 LR을 선형적으로 증가

epoch = 1000 : 전체 데이터의 총 학습 횟수

clip = 1.0 : Weight 값 범위 지정

weight_daecay = 5e-4 : 과적합 방지를 위한 가중치 감쇠값

inf = float('inf') : 무한값

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

학습 결과

BLEU(Bilingual Evaluation Understudy)

: 사람이 해석한 것(reference)과기계가 해석한 것(hypothesis)간에 일치율을 점수화한 것으로 번역의 품질을 평가한다.

BLEU 작동 방식

1. N-gram일치 : 번역된 문장과 참조 문장간의 일치 측정
2. 정밀도 : 번역된 문장에서 참조 문장과 일치하는 N-gram 비율 계산
3. 브레비티 패널티 : 번역된 문장이 참조 문장보다 짧을 경우, 페널티 적용
4. 최종 점수 : 각 N-gram의 정밀도를 기하평균으로 계산하고 브레비티 페널티를 적용하여 최종 BLEU 점수 계산

BLEU 점수는 0에서 1 사이의 값을 가지며, 1에 가까울수록 번역의 품질이 높다는 것을 의미하며 보통 백분율로 표현한다.

에러를 해결하지 못해 실제 학습은 할 수 없었지만 **Attention All You Need** 논문에서 모델의 파라미터를 변경해가면서 성능에 어떤 영향을 주는지 실험한 내용을 확인해보았다.

영어에서 독일어로의 번역 성능 변화를 확인한 결과 단일 헤드 어텐션일때 BELU 점수가 낮아지며 너무 많은 헤드를 사용할 때도 품질이 저하되는 것을 확인하였다. 또한 모델이 클 수록 성능이 좋아지고 drop-out이 과적합을 방지하는 데 도움이 된다는 것을 확인하였다.